

大语言模型技术原理深度解析

第一章：引言——从统计语言模型到大语言模型的演进

1.1 语言建模的本质

语言建模是自然语言处理（Natural Language Processing, NLP）领域的核心任务之一。其基本目标可以表述为：给定一个词序列

$w_1, \dots, w_{t-1}$
，预测下一个词
的概率分布

$P(w_t | w_1, \dots, w_{t-1})$

。这个看似简单的条件概率问题，实际上触及了人工智能领域最具挑战性的难题——如何让机器理解并生成人类语言。

早期的语言模型采用基于统计的方法。n-gram模型通过马尔可夫假设简化问题，认为下一个词的出现仅依赖于前面

n-1 个词。具体而言，一个三元模型（trigram）的预测公式为：

$$P(w_t | w_{t-2}, w_{t-1}) = \frac{\text{count}(w_{t-2}, w_{t-1}, w_t)}{\text{count}(w_{t-2}, w_{t-1})}$$

其中

count

$\text{count}(\cdot)$ 表示在训练语料中统计得到的共现频次。这种基于频次统计的方法虽然直观且计算效率高，但存在两个根本性缺陷：其一是数据稀疏性问题，即使在大规模语料中，大部分可能的 n-gram 组合从未出现过；其二是无法捕捉词语之间的语义相似性——它无法理解“猫”和“猫咪”在语义上的相近关系。

1.2 神经网络语言模型的兴起

2003年，Bengio等人提出了神经概率语言模型（Neural Probabilistic Language Model, NPLM），开创性地使用前馈神经网络来学习词的分布式表征。其核心思想是将每个词映射到一个低维稠密的实数向量（即词嵌入，word embedding），然后通过神经网络学习这些向量之间的组合规律。这一范式转变带来了决定性的突破：语义相似的词在向量空间中彼此邻近，模型因此具备了泛化能力——即使在训练中未见过的词组，如果其组成词的向量已在相似上下文中出现过，模型仍能做出合理预测。

词嵌入的学习过程本身也极具启发性。以著名的Word2Vec模型为例，其连续词袋（Continuous Bag-of-Words, CBOW）架构试图从上下文词汇的向量和来预测中心词；而Skip-gram架构则正好相反，它从中心词出发预测其周边的上下文词汇。训练完成后，这些向量空间展现出了惊人的规律性。最为人熟知的案例是：

$$\text{vec}(\text{"king"}) - \text{vec}(\text{"man"}) + \text{vec}(\text{"woman"}) \approx \text{vec}(\text{"queen"})$$

这一向量运算关系表明，神经网络并非简单地记忆词频统计，而是在内部形成了对词汇语义关系的高阶抽象表征。

1.3 注意力机制的诞生与Transformer架构

尽管循环神经网络（Recurrent Neural Network, RNN）及其变体长短期记忆网络（Long Short-Term Memory, LSTM）和门控循环单元（Gated Recurrent Unit, GRU）在序列建模上取得了显著进展，但它们固有的顺序计算特性带来了两大问题：其一是难以并行化，长序列的训练效率低下；其二是长距离依赖问题，信息在逐时间步传递过程中会逐渐衰减或膨胀，即所谓的梯度消失或梯度爆炸。

2017年，Google团队在论文《Attention Is All You Need》中提出了Transformer架构，彻底改变了这一局面。Transformer摒弃了循环结构，仅依靠注意力机制（Attention Mechanism）捕捉序列中任意两个位置之间的依赖关系。其核心操作是缩放点积注意力（Scaled Dot-Product Attention）：

$$\text{Attention}(\text{softmax}(\text{Attention}(Q, K, V)))$$

其中

Q （Query）、

K （Key）、

V （Value）均为输入序列经过不同线性变换得到的矩阵。公式的核心逻辑可以解读为：对于每个查询向量，计算它与所有键向量的点积相似度，除以

进行缩放以防止点积值过大导致softmax梯度饱和，然后通过softmax归一化为注意力权重，最后用这些权重对值向量进行加权求和。

为了捕捉不同层面的特征交互，Transformer引入了多头注意力（Multi-Head Attention）机制。它将

V 分别线性投影到

h 个不同的子空间，在每个子空间中独立执行注意力计算，然后将

h 个输出拼接起来再次线性变换：

MultiHead

Concat

head

head

MultiHead(Q,K,V)=Concat(head

,...,head

head

Attention

head

=Attention(QW

,KW

,VW

这种设计使得模型能够同时从"语义角色"、"词性搭配"、"句法结构"等多种角度审视序列中的关系，极大增强了表达能力。

Transformer的另一项关键设计是位置编码（Positional Encoding）。由于注意力机制本身不具备感知顺序的能力，必须在输入中注入位置信息。原始Transformer使用正弦和余弦函数构造固定位置编码：

sin

10000

model

(pos,2i)

=sin(

10000

2i/d

model

pos

cos

10000

model

(pos,2i+1)

=cos(

10000

2i/d

model

pos

其中

pos 是位置索引，

i 是维度索引。这种设计使得模型可以轻易学到相对位置关系，因为对于任意固定偏移量

$pos+k$

可以表示为

pos

的线性函数。

大语言模型技术原理深度解析

第二章：大规模预训练的核心技术

2.1 预训练范式与自监督学习

大语言模型的成功离不开预训练（Pre-training）与微调（Fine-tuning）的两阶段范式。在预训练阶段，模型在海量无标注文本上通过自监督学习（Self-supervised Learning）获取通用的语言知识和世界知识；在微调阶段，模型在少量有标注的特定任务数据上进行调整，以适应下游的具体应用。

自监督学习的核心在于从数据内部构造监督信号，无需人工标注。对于语言模型而言，最经典的自监督任务是自回归语言建模（Autoregressive Language Modeling）：给定前文，预测下一个词。其训练目标函数为最大化对数似然：

$$\log \prod_{t=1}^T P(w_t | w_{1:t-1}; \theta)$$

其中

θ 表示模型的参数，

t 表示位置

t 之前的所有词。这一目标函数驱使模型学习理解上下文并生成连贯的文本。

另一种广泛使用的预训练任务是掩码语言建模（Masked Language Modeling, MLM），由BERT模型率先推广。在该任务中，输入序列中随机选取15%的词元（token）进行掩码处理，模型需要根据未被掩码的上下文预测被掩码位置的原始词元。MLM的损失函数可以表示为：

$$\text{MLM Loss} = - \frac{1}{|M|} \sum_{i \in M} \log P(w_i | w_{-i}; \theta)$$

其中

M 是被掩码的位置集合，

表示除掩码位置外的所有上下文词元。相比于自回归建模，MLM允许模型同时利用左侧和右侧的上下文信息（即双向上下文），在自然语言理解类任务上表现更优。然而，MLM在预训练和下游任务之间存在差异——下游任务中通常没有掩码符号，这种不一致性被称为“预训练-微调差异”（Pre-train Fine-tune Discrepancy）。

序列到序列（Seq2Seq）预训练则结合了两范式的优点，在编码器部分使用MLM（双向上下文理解），在解码器部分使用自回归建模（单向生成）。典型代表是T5（Text-to-Text Transfer Transformer）和BART。这类模型特别适用于翻译、摘要、问答等输入输出均为文本的任务。

2.2 Transformer架构的规模化

大语言模型的“大”体现在多个维度上，其中最关键的是三个规模参数：

维度 符号 含义 典型值（GPT-3）

层数

L Transformer块的堆叠数量 96

隐层维度

model

model

每个位置的向量表示维度 12288

注意力头数

h 多头注意力中的并行头数 96

这三个参数共同决定了模型的参数量。对于一个标准的Transformer解码器模型，其总参数量

N 可以近似估算为：

model

（自注意力层与前馈层）

词嵌入层参数量

$N \approx 12Ld$

model

（自注意力层与前馈层）+词嵌入层参数量

更精确的计算包括：每个Transformer块中包含多头注意力和前馈网络两个子层。多头注意力部分包含

V 三个权重矩阵（均为

model

model

model

model

）以及一个输出投影矩阵（同样

model

model

model

model

), 共

model

model

个参数。前馈网络通常包含两个线性变换，中间维度设置为

model

model

, 因此参数量约为

model

model

model

model

model

model

$\times(4d$

model

$)+(4d$

model

$)\times d$

model

$=8d$

model

。因此每个Transformer块的参数量约为

model

$12d$

model

。词嵌入层的参数量为

model

$V\times d$

model

, 其中

V 是词表大小。

以GPT-3为例, 当

$L=96,$

model

12288

model

$=12288$ (即 $12,288$),

50000

$V\approx 50000$ 时, 仅Transformer块的参数量就达到:

12288

150
994
944
174
 $96 \times 12 \times (12288)$
 $\approx 96 \times 12 \times 150,994,944 \approx 174 \times 10$

即约1740亿参数，再加上词嵌入层和其他组件的贡献，最终总参数量达到1750亿。这种规模的参数量意味着模型有机会在训练过程中学习和压缩海量的语言模式和知识片段。

2.3 扩展法则 (Scaling Laws)

OpenAI在2020年发表的《Scaling Laws for Neural Language Models》是一项里程碑式的研究。该研究发现，当模型的参数量

N、数据集规模

D (以token数量计) 和计算预算

C (以浮点运算次数计) 分别独立增加时，模型的测试损失

L 遵循稳定的幂律关系：

$L(N) \approx$

$, L(D) \approx$

$, L(C) \approx$

其中指数

0.076

$\approx 0.076,$

0.095

$\approx 0.095,$

0.050

≈ 0.050 。这些幂律关系的关键含义在于：

没有“免费午餐”：模型性能的增长需要相应的资源投入；只要投入不足，性能就会被瓶颈因素锁定。

最优资源分配：对于给定的计算预算

C，存在最优的参数量

N 和训练token数

D 的分配比例。研究发现，当

N 与

D 大致按照

0.73

$$N \propto C$$

0.73

0.27

$$D \propto C$$

0.27

的比例增长时，损失最小化。这意味着在计算预算有限时，增加模型规模比增加数据规模更有效——这一结论深刻影响了后续大模型的研发方向。

涌现能力：扩展法则主要适用于预测下一个词元（next token prediction）的损失。当模型规模超过某个阈值（通常在百亿参数以上）时，会出现一些“涌现能力”（Emergent Abilities）——这些小模型几乎不具备的、突然出现的高级能力，如上下文学习（In-context Learning）、思维链推理（Chain-of-Thought Reasoning）等。这些涌现能力不再遵循平滑的幂律扩展规律。

大语言模型技术原理深度解析

第三章：训练流程与优化技术

3.1 数据准备与预处理

大语言模型的训练数据是其能力的根基。典型的大模型训练语料规模达到数万亿token，涵盖网页文本、书籍、论文、代码、对话记录等多种来源。数据准备通常包含以下关键步骤：

数据清洗：原始文本中存在大量噪声，包括HTML标签、重复内容、模板生成的占位符文本、敏感信息等。常见的清洗策略包括：

基于启发式规则过滤：去除过短的行、特殊字符比例过高的文本、重复标点符号的段落

基于分类器的质量过滤：训练一个小型分类器，区分高质量文本（如维基百科、出版书籍）和低质量文本（如用户评论区的广告、乱码）

去重：使用MinHash或SimHash等局部敏感哈希（Locality Sensitive Hashing）算法，在大规模数据中快速识别并移除近似重复的文档

分词（Tokenization）：原始文本需要被切分为模型可以处理的离散单位。现代大模型普遍使用字节对编码（Byte-Pair Encoding, BPE）或其变体。BPE的核心算法如下：

text

Algorithm: Byte-Pair Encoding (BPE)

- 初始化词表为所有单个字符的集合
- 统计语料中所有相邻符号对的频次
- 将频次最高的符号对合并为一个新的符号，添加到词表中

4. 重复步骤2-3，直到词表大小达到预设阈值

BPE的优势在于可以平衡词表大小和编码效率：常见词汇被编码为一个token（例如“the”：1个token），不常见的词汇被拆分为子词（例如“lowercase”：["low", "er", "case"]），而完全未知的字符最终会退化为字节级别的表示，从而理论上可以处理任何Unicode字符。

近年来，Google的SentencePiece库和OpenAI的tiktoken成为主流分词器实现。SentencePiece不假设输入为分好词的文本（例如直接处理中文和英文混合文本），并在训练后可以无损地恢复原始句子。

3.2 分布式训练系统

训练一个千亿参数级别的语言模型需要巨大的计算资源。以GPT-3为例，1750亿参数的模型在约4500亿token上训练，总计算量达到约 3.14×10^{23} FLOPS。假设使用单张NVIDIA A100 GPU（理论峰值为312 TFLOPS的FP16矩阵运算），即使完全达到峰值效率，也需要约1000万秒，即超过115天。而实际训练中，由于通信开销、内存限制等因素，单卡训练是不可能的。

因此，分布式训练系统需要综合运用多种并行策略：

数据并行（Data Parallelism）：最简单的并行方式。将训练数据分片到多个GPU上，每个GPU维护一份完整的模型副本，独立计算梯度，然后通过All-Reduce通信原语同步梯度。数据并行的瓶颈在于：当模型太大无法放入单卡显存时，此方法失效。

模型并行（Model Parallelism）：将模型的不同部分分配到不同GPU上。包括：

流水线并行（Pipeline Parallelism）：将Transformer层按顺序切分到不同GPU。例如，GPU 0负责第1-24层，GPU 1负责第25-48层。前向传播时，GPU 0计算完成后将中间结果传递给GPU 1；反向传播时流程相反。流水线并行的问题是引入“GPU空闲时间”（bubble），但可以通过微批次（micro-batch）策略缓解。

张量并行（Tensor Parallelism）：将单个Transformer层内的矩阵运算切分到多个GPU。以多头注意力为例，可以将不同的注意力头分配到不同GPU上独立计算，最后汇总结果。张量并行需要高带宽的通信（如NVLink），在节点内效果较好。

ZeRO（Zero Redundancy Optimizer）：由微软DeepSpeed团队提出的优化方案。ZeRO对模型的三种状态进行分区存储而不是冗余存储：优化器状态（如Adam的一阶矩和二阶矩）、梯度、模型参数。ZeRO分为几个阶段：

Stage 1：优化器状态分区（通信量与数据并行相当）

Stage 2：优化器状态+梯度分区

Stage 3：优化器状态+梯度+模型参数全分区，允许训练超出单卡显存容量的模型

ZeRO-3通过动态收集和释放参数，实现了在1024个GPU上训练万亿级参数模型的能力，同时保持接近线性的扩展效率。

3.3 优化算法与稳定性

训练大语言模型面临着非平凡的优化挑战。传统随机梯度下降（SGD）及其动量变体在大模型上的表现往往不如自适应学习率方法。目前最主流的优化器是Adam（Adaptive Moment Estimation）及其变体AdamW。

Adam的核心思想是同时维护梯度的一阶矩（动量）和二阶矩（自适应学习率）：

$$\begin{aligned} & \mathbf{m}^{t-1} \\ & + (1 - \beta_1) \mathbf{g}^t \\ & \mathbf{m}^t = \beta_1 \mathbf{m}^{t-1} + (1 - \beta_1) \mathbf{g}^t \\ & \mathbf{v}^{t-1} \\ & + (1 - \beta_2) \mathbf{g}^t \odot \mathbf{g}^t \\ & \mathbf{v}^t = \beta_2 \mathbf{v}^{t-1} + (1 - \beta_2) \mathbf{g}^t \odot \mathbf{g}^t \end{aligned}$$

其中

$\mathbf{g}^t$ 是第t步的梯度，

β_1 是衰减系数（典型值0.9和0.999），

η 是学习率，

ϵ 是避免除零的小常数。

训练稳定性是大模型训练中尤为突出的问题。随着模型规模增大，训练过程更容易出现梯度爆炸或损失尖峰（loss spikes）。常用对策包括：

梯度裁剪（Gradient Clipping）：在更新参数前限制梯度的范数。如果

$\|\mathbf{g}\| > \tau$ ，则

$$\mathbf{g} \leftarrow \tau \cdot \frac{\mathbf{g}}{\|\mathbf{g}\|}$$

。

典型阈值

τ 设置为1.0。

学习率预热（Learning Rate Warmup）：在训练初期让学习率从一个很小的值线性增长到目标值。预热机制可以避免训练初期的参数剧烈震荡，帮助损失平稳下降。

权重衰减（Weight Decay）：在损失函数中加入

$$\lambda \|\boldsymbol{\theta}\|^2$$

正则项，鼓励参数值保持较小，改善泛化。

混合精度训练（Mixed Precision Training）：使用FP16进行前向和反向传播，同时使用FP32维护主参数和优化器状态。这不仅能减少显存占用（约一半），还能加速计算。但FP16的有限数值范围容易导致下溢，需要通过损失缩放（loss scaling）技术补偿。

3.4 微调与对齐

预训练后的模型擅长文本补全，但未必会以用户期望的方式回答问题。例如，用户问“如何制作某种危险品”，预训练模型可能会如实写出配方，但这显然不符合安全规范。为了让模型产生有用、安全、符合人类偏好的回复，需要进行微调和对齐。

监督微调 (Supervised Fine-Tuning, SFT)：在高质量的人工标注的指令-回复数据上进行监督学习。数据格式通常为：

text

Instruction: 写一首关于秋天的五言绝句

Response: 秋风起寒色，落叶满长安。登高望远路，归雁入云端。

SFT过程与预训练几乎相同，只是训练数据从海量无标签文本替换为少量（通常数万到数十万条）高质量指令数据。这一步教会模型遵循指令、遵循对话格式。

人类反馈强化学习 (Reinforcement Learning from Human Feedback, RLHF)：这是对齐阶段的进阶技术，由InstructGPT/ChatGPT首次大规模应用。RLHF包含三个步骤：

步骤1：收集偏好数据。对于同一个提示 (prompt)，让SFT模型生成多个回复（例如4个），人类标注员对这些回复进行排序或两两比较。数据格式示例：对于提示P，人类标注员认为回复A优于回复B。

步骤2：训练奖励模型 (Reward Model, RM)。奖励模型通常是一个与待对齐模型规模相近（或略小）的Transformer模型，它将回复r的最后一个特殊token的隐藏状态映射为一个标量奖励值

(P,r)。训练目标是让奖励模型对人类偏好数据的预测最大化：

$$\log \mathbb{E}_{(P,r) \sim D} [\log \sigma(R(P,r) - R(P,r'))]$$

其中

是获胜回复（人类偏好），

是失利回复，

σ 是sigmoid函数。

步骤3：通过强化学习优化策略。将SFT模型作为初始策略

SFT

SFT

，从中派生出待优化的策略

。目标是最大化奖励模型的评分，同时通过KL散度惩罚防止策略偏离原始模型太远（避免“奖励破解”（reward hacking）导致生成不自然的文本）：

$$\begin{aligned} & \text{SFT} \\ & P_{D,r} \pi \\ & (\cdot | P) \\ & (P,r) - \beta \cdot D \\ & (\cdot | P) \|\pi \\ & \text{SFT} \\ & (\cdot | P) \end{aligned}$$

在实现层面，近端策略优化（Proximal Policy Optimization, PPO）是最常用的强化学习算法。

直接偏好优化（Direct Preference Optimization, DPO）：2023年提出的新方法，绕过了显式训练奖励模型和强化学习的过程，直接将偏好数据转换为监督学习的形式。DPO推导出了一个可以利用偏好数据进行训练的封闭形式损失函数，大大简化了对齐流程。

大语言模型技术原理深度解析

第四章：高级能力与涌现现象

4.1 上下文学习（In-Context Learning）

上下文学习是大语言模型最令人惊叹的涌现能力之一。传统监督学习需要在标注数据上训练模型，使模型能够泛化到未见过的测试样本。而上下文学习完全跳过了参数更新步骤：只需在提示中提供几个示例（称为“示例演示”），模型就能理解任务模式并直接应用于新的输入。

上下文学习的标准格式可以表示为：

```
text
问题：法国的首都是什么？
答案：巴黎

问题：日本的首都是什么？
答案：东京

问题：巴西的首都是什么？
答案：
```

模型在接收到包含两个示例的提示后，能够推断出这是一个“首都问答”任务，并生成“巴西利亚”作为答案。其关键特征在于：

无需参数更新：模型的权重在推理过程中完全冻结

示例数量灵活：零样本（0-shot，无示例）、少样本（few-shot，2-10个示例）、多样本（数百个示例）均可

任务通用性：同一个模型可以处理翻译、摘要、代码生成、推理等完全不同类型的任务，仅通过提示内容切换

上下文学习的底层机制至今仍是活跃的研究课题。以下几种假说被广泛讨论：

元学习假说：预训练过程中的海量任务（不同文章、不同格式、不同主题）实际上在训练模型学习一种“通用学习算法”。当模型在提示中看到示例时，它会识别出这是一个需要从示例中推断模式的任务，并动态激活相应的内部计算路径。

任务识别假说：示例的作用不是让模型“学习”新任务，而是让模型在预训练记忆中找到与当前示例最匹配的任务模式。例如，模型在预训练中见过无数个QA格式的对话，示例仅仅充当了一个“路由信号”，告诉模型应该激活QA模块而非翻译模块。

梯度近似假说：Transformer内部的前向计算在高维空间中实际上近似了某种隐式的梯度下降。换句话说，上下文学习可以被理解为模型在内部隐式地“拟合”示例数据，并将这种拟合结果应用于查询。

4.2 思维链推理 (Chain-of-Thought Reasoning)

思维链推理是上下文学习的一个强大扩展。标准上下文学习要求模型直接输出最终答案，而思维链推理要求模型在输出答案之前，先输出一系列中间推理步骤。这种“慢思考”模式显著提升了模型在数学、逻辑推理和常识问题上的表现。

典型的思维链提示格式如下：

text

问题：李华买了3个苹果，每个2元；又买了5个橙子，每个3元。他一共花了多少钱？

思考：先计算苹果的总价： $3 \times 2 = 6$ 元。再计算橙子的总价： $5 \times 3 = 15$ 元。然后相加： $6 + 15 = 21$ 元。

答案：21元

问题：一个长方形的长是8米，宽是5米。它的面积是多少平方米？

思考：长方形面积 = 长 $\times$ 宽 = $8 \times 5 = 40$ 平方米。

答案：40平方米

问题：一家书店有120本书，第一天卖出1/3，第二天卖出剩下的1/4，还剩多少本书？

思考：

模型需要输出推理链才能得到最终答案。思维链推理之所以有效，基于以下几点原因：

分解复杂问题：将多步推理分解为一系列中间步骤，每一步的计算复杂度较低

减少错误累积：如果模型随机猜测答案，错误的概率随问题复杂度指数增长；而分解后的每个步骤中错误概率相对可控

可验证性：用户可以看到推理过程，判断答案是否可信（类似数学考试中“列出计算过程”的要求）

研究发现，思维链能力仅在模型规模超过约1000亿参数时才会显现。规模较小的模型（如小于100亿参数）即使被明确要求“逐步思考”，也往往无法生成有效的推理链条，或者生成的链条与答案不一致。这被认为是“涌现能力”的典型示例。

思维链的一个改进版本是自洽性思维链（Self-Consistency with CoT）：让同一个模型多次生成不同的推理路径和答案（使用非零的温度参数增加随机性），然后通过投票机制选择最一致的答案。自洽性可以将数学推理的错误率降低30%-50%。

4.3 指令遵循与泛化

指令遵循能力使大语言模型能够理解并执行用自然语言描述的任务，而不需要严格的格式要求。例如：

“把下面这句话翻译成法语”（模糊指令）

“用三个要点总结这篇文章”（结构化指令）

“假设你是一个历史老师，解释一下法国大革命的原因”（角色扮演指令）

指令遵循能力的获得主要来自两个阶段：

1. 指令微调（Instruction Tuning）：在包含数千到数十万条（指令，输出）对的数据集上进行监督微调。关键挑战在于指令的多样性——数据需要涵盖不同领域（科学、历史、娱乐）、不同难度、不同格式的任务。FLAN、T0、Super-NaturalInstructions 等数据集为此提供了基础。

指令微调的核心观察是任务泛化：模型在100个任务上微调后，可以泛化到从未见过的100个新任务上。这说明模型不是简单地记忆“某个指令对应某个输出”，而是学习到了“按照自然语言描述执行操作”的通用技能。

2. 对话格式对齐：通过RLHF或DPO等技术，模型进一步学会理解对话的隐含意图、处理多轮上下文、以及在不确定时主动询问澄清信息。这一阶段的训练数据通常是人工标注的高质量对话。

4.4 知识容量与事实召回

大语言模型在预训练阶段将规模庞大的知识压缩到其权重中。GPT-3拥有1750亿参数，理论上可以存储大约40-50GB的压缩信息，相当于约2000万页文本的精华。这种“隐性知识库”的运作涉及以下机制：

知识存储：研究表明，事实知识主要存储在Transformer的前馈网络（Feed-Forward Network, FFN）层中。每个FFN层可以被看作一个键值存储系统：第一层线性变换将输入向量映射为“键”，激活函数（通常是GELU或ReLU）选择一部分键，第二层线性变换将这些键映射回值向量。知识检索时，模型通过查询与键的匹配程度决定提取哪些事实。

语言模型偏差：尽管存储了大量知识，模型在回答事实性问题时仍可能出错。常见偏差包括：

频率偏差：预训练中出现频率高的事实更容易被正确召回（例如“姚明打篮球”比“姚明在2009年做了脚部手术”更稳定）

近因偏差：训练数据中后期出现的信息覆盖早期信息

虚假相关性：模型可能基于错误的统计相关性虚构事实（即“幻觉”）